Basics

Matrix addition:

$$\begin{bmatrix} 4 & 8 \\ 3 & 7 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 5 & 2 \end{bmatrix} = \begin{bmatrix} 5 & 8 \\ 8 & 9 \end{bmatrix}$$

```
~~int sum = 0~~;
int[][] arr3 = new int[row][col]

for(i=0; i<row-1; i++){
    for(j=0; j<col-1; j++){
        arr3[i][j] = arr1[i][j] + arr2[i][j];
    }
}
```

Matrix multiplication:

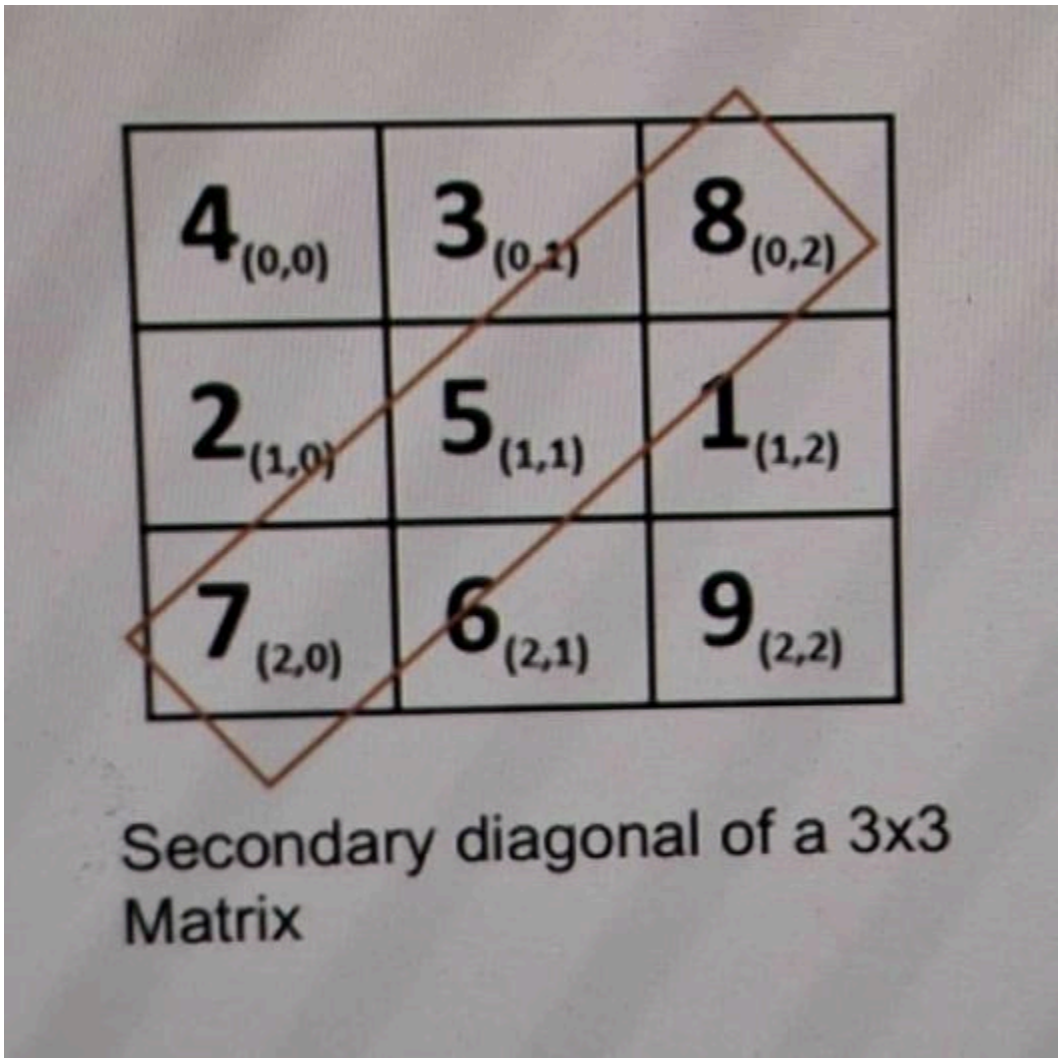
$$\begin{array}{c} \begin{array}{cc} 0 & 1 \end{array} \\ \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}_{m\times n} \end{array} \times \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}_{n\times p} = \begin{bmatrix} 1\times5+2\times7 & 1\times6+2\times8 \\ & \end{bmatrix}_{m\times p}$$

i fix
j change

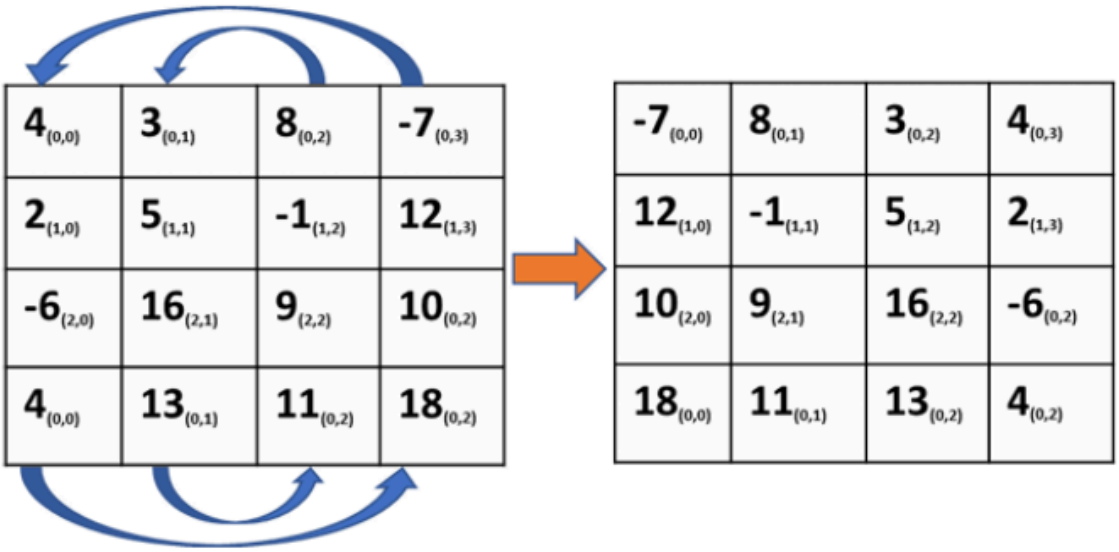
i change
j fix

```
int row1 = mat1.length;      ~~int~~
int col2 = mat2[0].length;

~~int r~~ int[][] result = ~~new int[][];~~ new int[row1][col2]

for(int i=0; i<row-1; i++){
    for(int j=0; j<col-1; j++){
        for(int k=0; k<col1-1; k++){
            result[i][j] += mat1[i][k] * mat2[k][j];
        }
    }
}
```

Remainder: Use 1 loop for main diagonal sum




```
for(int i=0; i<row: i++){
   sum= sum+arr[i][col-i-1];
}
```

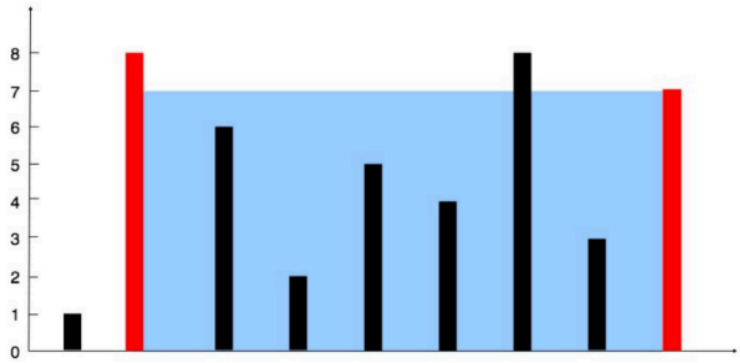



```
for (int i = 0; i < row; i++) {
    for (int j = 0; j < col / 2; j++) {
        int temp = arr[i][j];
        arr[i][j] = arr[i][col - 1 - j];
        arr[i][col - 1 - j] = temp;
    }
}
```

---

1. Container with Most Water [5 marks]

You are given an integer array called ***height*** of length n. You can imagine the values of the array as vertical lines drawn. If you imagine the y-axis and the x-axis then you can say the two endpoints of the ith vertical line are (i, 0) and (i, height[i]) .
Find such two lines that together with the x-axis form a container, such that the container



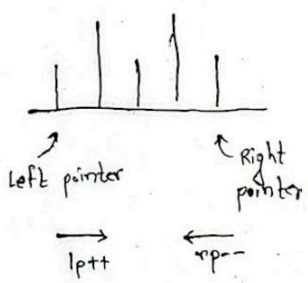
contains the most water.
Return the maximum amount of water a container can store.

Note: that you may not slant the container.

Sample Given Array	Sample Output
height = [1,8,6,2,5,4,8,3,7]	49

Reverse, Palindrome

Optimal approach (Two pointer approach):



* container = $\omega \times ht$
  ($r-l$) min(h[l], h[r])

* min height ক move করবো

```
lp = 0 , rp = n-1
while (lp < rp){
    ω = rp - lp
    height = min(h[lp], h[rp])
    area = ω * h;
    maxWater = max(mW, area);
    if (h[lp] < h[rp]){
        lp++;
    }
    else{
        rp--;
    }
}
```

Spiral Matrix :

① 1 → 2 → 3 → 4
↓
8 ← 7 ← 6 ← 5
↓
9 → 10 → 11 → 12

```
for (int i = 0 ; i < row; i++){
    if (i % 2 == 0){
        for (j = 0; j < col ; j++){
            S.O.P (arr[i][j] + "->");
        }
    }
    else {
        for (j = col - 1 ; j >= 0 ; j--){
            S.O.P (arr[i][j] + "->")
        }
    }
```

②

1 ← 2 ← 3 ← 4
↓
8 → 7 → 6 → 5
↓
9 ← 10 ← 11 ← 12

~~for (i = arr.length-1 ; i >= 0 ; i--){~~

```
for (int i = 0 ; i < row; i++){
    if (i % 2 == 0){
        for (j = col-1 ; j >= 0 ; j--){
            S.O.P (arr[i][j] + "->");
        }
    }
    else {
        for (j = 0 ; j < col ; j++){
            S.O.P (arr[i][j] + "->");
        }
    }
```

③

1 ← 2 ← 3 ← 4
↑
5 → 6 → 7 → 8
↑
9 ← 10 ← 11 ← 12

```
for (i = row-1; i >=0; i--){
  if( (row-1-i) % 2 == 0){
      for(j = col-1; j>=0; j--) {
          s.o.p(arr[i][j]);
      }
  }

  else{
      for(j=0; j<col; j++){
          s.o.p(arr[i][j]);
      }
  }
```

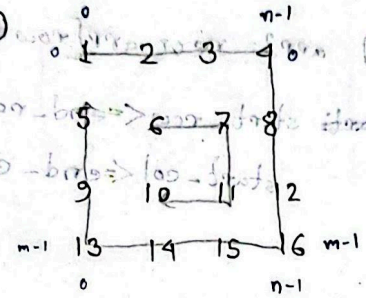
---

④

1 ← 2 3 ← 4
↓ ↑ ↓ ↑
5 6 7 8
↓ ↑ ↓ ↑
9 10 ← 11 12

```
for(j = col-1; j>=0; j--){
  if ((col-1-j) % 2 == 0){
      for(i = row-1; i>=0; i--){
          sop(arr[i][j]);
      }
  }

  else{
      for(i=0; i<row; i++){
          sop(arr[i][j]);
      }
  }
```

⑤



Top
Right
Bottom
Left

start_row = 0 start_col = 0
end_row = m-1 end_col = n-1

① Top: for (j = scol to ecol)
arr [srow] [j]

② Right: for (i = start_row + 1 to end_row)
~~mat~~ arr [i] [end_col]

③ Bottom: for (j = end_col - 1 to start_col)
arr [end_row] [j]

④ Left: for (i = end_row - 1 to start_row + 1)
arr [i] [start_col]

start_row ++ start_col ++
end_row -- end_col --

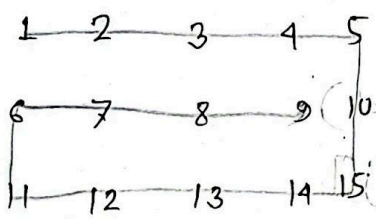
① Logic boundaries

② while condition

③ Corner case

```
int[] arr2 = new arr[row * col];
while (~~start~~ start_row <= end_row &&
        start_col <= end_col)
```

Corner case:




```
//top
//bottom  if (start_row == end_row) {
            break;
        }
// right
// left    if (start_col == end_col){
            break;
        }
```

Lab Task

1. Merge Sorted Arrays

You are given two integer arrays arr1 and arr2, sorted in non-decreasing order (ascending order). Merge arr1 and arr2 into a single array sorted in non-decreasing order (ascending order) and return the new sorted array.

[Try to solve it in an optimized way]

Sample Given Arrays	Sample Returned Array
arr1 = [1,2,3] arr2 = [2,5,6]	[1,2,2,3,5,6]
arr1 = [1,3,5,11] arr2 = [2,7,8]	[1,2,3,5,7,8,11]

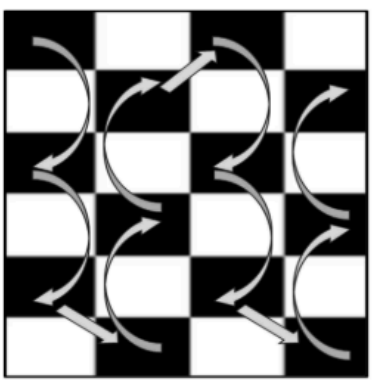
```
//Task 1
import java.util.Arrays;
public class merge {
   public static void main(String[] args){
      int arr1[]={1,2,3};
      int arr2[]={2,5,6};

      int[] arr=new int[arr1.length + arr2.length];

      int index=0;
      int i1=0;
      int i2=0;

      while(i1<arr1.length && i2<arr2.length){
         if(arr1[i1]< arr2[i2]){
            arr[index]=arr1[i1];
            i1++;
            index++;
         }
         else{
            arr[index]=arr2[i2];
            i2++;
            index++;
         }
      }
```

```
        while(i1<arr1.length){       //Remaining ones
            arr[index++]=arr1[i1++];
        }
        while(i2<arr2.length){
            arr[index++]=arr2[i2++];
        }

        System.out.println(Arrays.toString(arr));
    }
}
```

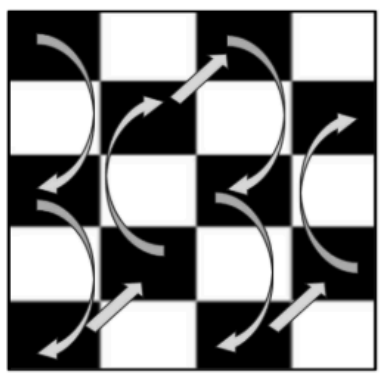
---

2. Zigzag Walk:

As a child, you often played this game while walking on a tiled floor. You walked avoiding a certain color, for example white tiles (it almost felt like if you stepped on a white tile, you would die!). Now you are in a room of m x n dimension. The room has m*n black and white tiles. You step on the black tiles only. Your movement is like this:



OR



Now suppose you are given a 2D array which resembles the tiled floor. Each tile has a number. Can you write a method that will print your walking sequence on the floor?

Hint 1: The first tile of the room is always black.
Hint 2: Look out for the number of rows in the matrix Notice the transition from column 0 to column 1 in the above figures

Sample Given Matrix	Sample Output
3 8 4 6 1 7 2 1 9 3 9 0 7 5 8 2 1 3 4 0 1 4 2 8 6	3 9 1 1 2 4 7 2 4 9 1 8 6

	j=0	j=1	j=2	j=3
i=0	B	W	B	W
i=1	W	B	W	B
i=2	B	W	B	W
i=3	W	B	W	B

//2

```
public static void zigzagWalk(int[][] matrix) {
    int row = matrix.length;
    int col = matrix[0].length;

    for (int j = 0; j < col; j++) {
        if (j % 2 == 0) {                          //If for up to down
            for (int i = 0; i < row; i++) {
                if ((i + j) % 2 == 0) {            //If to skip
                    System.out.print(matrix[i][j] + " ");
                }
            }
        }


        else {
            for (int i = row - 1; i >= 0; i--) {
                if ((i + j) % 2 == 0) {
                    System.out.print(matrix[i][j] + " ");
                }
            }
```

```
        }
        System.out.println();
    }
  }
}
```

---

3. Decryption Process:

Suppose you're working as a cryptographer for a secret intelligence agency. You are given an encrypted matrix as an input. You have to decrypt it after some processing of the given matrix.

To decrypt this message efficiently, you have a task to construct a method called **decrypt_matrix(matrix)** which takes an encrypted matrix as input and returns a decrypted linear array. The process of finding the decrypted linear array is given below:

You have to find out the column-wise summations for each column and store the difference of subsequent column-wise summations in a new linear array.

Sample Given Matrix	Sample Output	Explanation
1 3 1 6 4 2 5 1 7 9 3 3 8 5 4	-13 1	Sum of 0th column = 29 Sum of 1st column = 16 Sum of 2nd column = 17 Therefore, the size of the resulting array is 2 and the array is: 16-29 = -13 17-16 = 1

```
//3
import java.util.Arrays;
public class decryption {
   public static void main(String[] args){
      int[][] matrix={
         {1, 3, 1},
         {6, 4, 2},
         {5, 1, 7},
         {9, 3, 3},
         {8, 5, 4}
```

```
        };
        decrypt_matrix(matrix);
    }



    public static void decrypt_matrix(int[][] arr){
        int col= arr[0].length;

        int[] arrS=new int[col]; //To store sum
        int sum=0;
        int indxS=0;

        int[] arrD=new int[col-1]; //To store decrypted
        int indx=0;


        for(int j=0; j<col; j++){
            sum=0;
            for(int i=0; i<arr.length; i++){
                sum+=arr[i][j];
            }
            arrS[indxS++]=sum;
        }


        for(int k=1; k<arrS.length; k++){
            arrD[indx++]=arrS[k]-arrS[k-1];
        }


        System.out.println(Arrays.toString(arrD));
    }

}
```

Lab Assignment

//Task 1 **(Row Rotate)**

1. Row Rotation Policy of Your Classroom [5 marks]

You are no longer permitted to choose your own seat on the new campus of your university. You must abide by the new regulations, which state that if you sit in the first row for the first week, you must shift to the second row for Week2, the third row for the Week3, and so on. In your classroom, there are a total of 6 rows and 5 columns. Your friend "AA" wants to know from you that on the upcoming exam week in which row he will be in. Your task is to implement a function `row_rotation(exam_week, seat_status)` that takes the exam_week and a 2D array of current seat status as input and returns the row number in which your friend "AA" will be seated and print the seat status for that week.

Sample User Input & Given Matrix	Output
User Input: exam_week = 3 Given Matrix: current_seat_status = | A | B | C | D | E | | F | G | H | I | J | | K | L | M | N | O | | P | Q | R | S | T | | U | V | W | X | Y | | Z | AA | BB | CC | DD |	| U | V | W | X | Y | | Z | AA | BB | CC | DD | | A | B | C | D | E | | F | G | H | I | J | | K | L | M | N | O | | P | Q | R | S | T | Your friend AA will be on row 2
Explanation: According to the example the exam is 3 weeks away from the current seat status. So, during the exam day, the row containing AA will move 2 times (basically the whole matrix will rotate downward vertically).	

```
public static Integer rowRotation( Integer examWeek, String[][] matrix ){
    int row=matrix.length;
    int col=matrix[0].length;

    int day= 1;
    int position= 0;

    //Move down until the examweek
    while(day<examWeek){
      String[] temp= matrix[row-1];
      for(int i=row-1;  i>0;  i--) {
        matrix[i] =  matrix[i-1];
       }
      matrix[0] =  temp;
      day++;
    }


   //print & finding AA
    for(int k=0; k<row; k++){
```

```
        for(int j=0; j<col; j++) {
            System.out.print(matrix[k][j]+ " | ");
              if(matrix[i][j].equals("AA")){
                  position= i+1;
             }
        }
        System.out.println();
    }


  return  position;
 }
```

//2

2. Matrix Compression [5 marks]

Complete the function **compress_matrix** that takes a 2D array as a parameter and return a new compressed 2D array. In the given array the number of row and column will always be even. **Compressing a matrix means grouping elements in 2x2 blocks and sums the elements within each block. Check the sample input output for further clarification.**

Hint: Generally the block consists of the (i,j), (i+1,j), (i,j+1) and (i+1, j+1) elements for 2x2 blocks.

You cannot use any built-in function except len() and range(). You can use the np variable to create an array.

Python Notation	Java Notation

import numpy as np
def compress_matrix (mat):
  # To Do
	public int[ ][ ] compress_matrix (int[] [] mat) {
	  // To Do
	}

Sample Input array	All Box (No need to create these arrays)	Returned Array	Explanation
[[1, 2, 3, 4], [5, 6, 7, 8], [1, 3, 5, 2], [-2, 0, 6,-3]]	[[1, 2], [[3, 4], [5, 6]] [[7, 8]] [[1, 3], [[5, 2], [-2, 0]] [[6, -3]]	[[14, 22], [2, 10]]	[[1+2+5+6, 3+4+7+8], [1+3+-2+0, 5+2+6+-3]]

//2

```
public static Integer[][] compressMatrix( Integer[][] matrix ){
    int row= matrix.length;
    int col=matrix[0].length;

    Integer[][] comp_matrix= new Integer[row/2][col/2];
    int row_indx;
```

```
    int col_indx;

    for(int i=0; i<row;i+=2){
        for(int j=0; j<col; j+=2){
            row_indx=i/2;
            col_indx=j/2;
          comp_matrix [row_indx][col_indx]=matrix[i][j]+ matrix[i+1][j] + matrix[i][j+1]+
matrix[i+1][j+1];
          }
        }

    return comp_matrix;
  }
```

//3

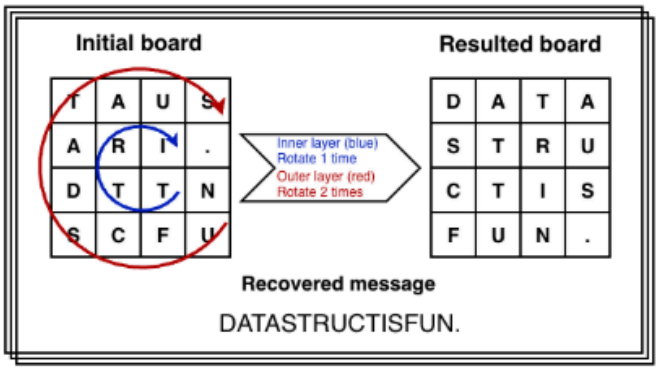
Sample Given Matrix 1	Sample Output 1
| 0 | 2 | 2 | 0 | | 50 | 1 | 2 | 0 | | 2 | 2 | 2 | 0 | | 1 | 100 | 2 | 0 |	Points Gained: 6. Your team is out.
Explanation: Player with value 50 has 2 in the cell below him (1 cell). Player with value 100 has 2 in the cell above and in the right cell (2 cells). So in total, they got (1+2)*2 = 6 points which was not enough to survive the game.	
Sample Given Matrix 2	**Sample Output 2**
| 0 | 2 | 2 | 0 | 2 | | 1 | 50 | 2 | 1 | 100 | | 2 | 2 | 2 | 0 | 2 | | 0 | 200 | 2 | 0 | 0 |	Points Gained: 14. Your team has survived the game.
Explanation: Player with value 50 has 2 in the cell above, cell below and the right cell (3 cells). Player with value 100 has 2 in the cell above and the cell below (2 cells). Player with value 200 has 2 in the above cell and right cell (2 cells). So in total, they gained (3+2+2)*2 = 14 points and survived the game. **Note:** For the cell with value 2 that is common between 2 players in position (2,1), both gained 2 points each, so it's not like if one player already added those 2 points, another player cannot. In the Given Matrix 2, Player with value 50 and player with value 200 are sharing a common 2 but both of them got points.	

```
public static void playGame( Integer[][] arena ){
        int row= arena.length;
        int col=arena[0].length;
        int sum=0;
```

```
for(int i=0; i<row; i++){
    for(int j=0; j<col;j++){
        if(arena[i][j]%50==0 && arena[i][j]!=0){

            //Up Check
            if(i-1>=0){                          //For corner case
              if(arena[i-1][j]==2){
                sum+=2;
                }
              }

            //Down CHeck
            if(i+1<=row-1 && arena[i+1][j]==2){
                sum+=2;
                }


            //Left Check
            if(j-1>=0 && arena[i][j-1]==2){
                sum+=2;
                }

            //Right Check
            if(j+1<=col-1 && arena[i][j+1]==2){
                sum+=2;
                }

            }
        }
    }

    System.out.print("Points Gained: " + sum + ".");
    if(sum>=10){
        System.out.println("Your team has survived the game.");
    }

    else{
        System.out.println("Your team is out.");
    }
}
```

//4

Space Complexity of the solution must be O(1) [can't create new array]



Sample Given Matrix 1	After Processing	Output

board = {
{'T','A','U','S'},
{'A','R','I','.'},
{'D','T','T','N'},
{'S','C','F','U'}
}
	board = {
	{'D','A','T','A'},
	{'S','T','R','U'},
	{'C','T','I','S'},
	{'F','U','N','.'}
	};
		DATASTRUCTISFUN.

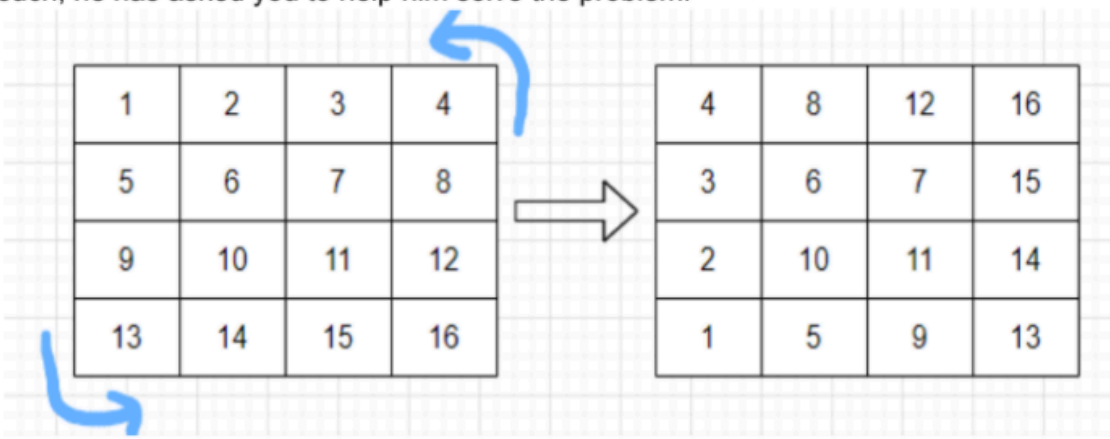
Explanation		
1 clockwise rotation on the innermost (2×2) layer, and 2 clockwise rotations on the outer (4×4) layer, each letter shifts to its correct position after these rotations. Then the message from the board is printed.		

```
public static void rotateSecret( Character[][] board ){
    int row=board.length;
    int col=board.length;

    int total_layer= row/2;

    for(int i=0; i<total_layer; i++){          //Duita layer r kaj korar jonno

      int no_rotation=0;
      while(no_rotation<total_layer-i){      //Outer layer gula extra time rotate r jonno
      rotate(board, i);
      no_rotation++;
      }
    }
  }
```

```
//Helper method
    public static void rotate(Character[][] board, int layer){
        int row=board.length;
        int col=board[0].length;

        int start_row=layer;
        int end_row= (row-1)-layer;
        int start_col=layer;
        int end_col= (col-1)-layer;

        Character temp = board[start_row][start_col];    //1st one stored


        //Left
        for(int i=start_row; i<end_row; i++){
            board[i][start_col]= board[i+1][start_col];
        }

        //Bottom
        for(int j=start_col; j<end_col; j++){
            board[end_row][j]= board[end_row][j+1];
        }

        //Right
        for(int i=end_row; i>start_row; i--){
            board[i][end_col]= board[i-1][end_col];
        }


        //Top
        for(int j=end_col; j>start_col+1; j--){
            board[start_row][j]= board[start_row][j-1];
        }

        board[start_row][start_col+1]=temp;
    }
```

RFTS Practice Sheet

Bob has already learnt about 2D matrices and its operations in the data structures course. However, he is currently facing problems in rotating a 2D square matrix. As such, he has asked you to help him solve the problem.




```
// Step 1: Transpose the matrix
    for (int i = 0; i < n-1; i++) {
        for (int j = i+1; j < n-1; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }

    // Step 2: Reverse each row
    for (int i = 0; i < n/2; i++) {
        for (int j = 0; j < n ; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[n-i-1][ j];
            matrix[n-i-1][j] = temp;
        }
    }
```

//Transpose following secondary diagonal matrix to do Counter clockwise

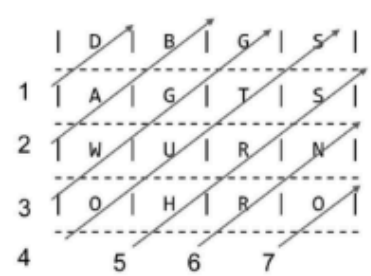
1. Transpose

2. Row shift

```
for(int i=row-1; i>0; i--){
   for(int j=i+1; j<col;j++){
          temp= arr[i][j];
          arr[i][j]=arr[row-1-j][col-1-i];
         arr[row-1-j][col-1-i]=temp;
}
```

You have hidden a password into a square matrix for your friend. Only you know that the password can be achieved through traversing the array in a zigzag fashion. Now help your friend develop a function/method that will take the 2D array as input and print the password in the output console.

Complete the function zig_zag which will take a NxN 2D array and print all the values in a zigzag pattern. Each value will be separated by a space. Assume the function will always receive a NxN 2D array. Your code should work for any NxN array.

Input Array	Output	Zigzag traversal explanation
| D | B | G | S | | A | G | T | S | | W | U | R | N | | O | H | R | O |	D A B W G G O U T S H R S R N O	

Hint: You need to follow the arrowed line 1 to line 7 to find out the zigzag pattern. Use one loop to print the upper triangle (line 1 to 4) and another loop to print the lower triangle (line 5 to 7).

```
// Lower half (including main diagonal)
      for (int row = 0; row < matrix.length; row++) {
         int i = row;
         int j = 0;
         while (i >= 0 && j <matrix.length) {
            System.out.print(matrix[i][j] + " ");
            i--;
            j++;
```

```
        }
    }

// Upper half (excluding main diagonal)
    for (int col = 1; col < matrix.length; col++) {
        int i = matrix.length - 1;
        int j = col;
        while (i >= 0 && j < matrix.length) {
            System.out.print(matrix[i][j] + " ");
            i--;
            j++;
        }
```

You are given a **9x5** dimensional 2D array which is used as a combination lock of a safe. You can independently rotate each column either upward or downward. You need to write a function that will take the 2D array and combination of the safe as an 1D array of length 5 and unlock the safe. To unlock the safe you need to make the **middle row (4th row** as rows start from 0) **same as the input 1D array.**

Note that the values of each column are unique and in the range of 1 to 9. To equate the middle row with the given combination (1D array), you need to perform **column rotation operation** (Just like an original safe). You may write and use helper functions.

The space complexity of your solution should be O(1) that means you cannot create any new array. **Finally, return the resulting array. In addition, write the Time complexity of your solution in terms of Big O notation.**

Input Array

2	8	9	6	7
4	2	5	8	5
6	7	1	4	3
9	6	7	2	9
7	**9**	**3**	**5**	**6**
8	1	6	9	2
5	4	2	7	1
3	3	8	1	4
1	5	4	3	8

combination:

1	2	3	4	5

Returned Array

7	4	9	1	1
8	3	5	3	4
5	5	1	6	8
3	8	7	8	7
1	**2**	**3**	**4**	**5**
2	7	6	2	3
4	6	2	5	9
6	9	8	9	6
9	1	4	7	2

Explanation

The returned array is rotated column wise in such a way so that the 4th row and the combination array have the same sequence of values.

Column wise rotation may be done upward or downward. No need to implement both.

```
//Find position
for (int j = 0; j < col; j++) {
    int position = -1;
    for (int i = 0; i < row; i++) {
        if (arr[i][j] == arr1[j]) {
            position = i;
            break;
        }
    }

    // Rotate downward until the target reaches middle row (4)
    while (position != 4) {
        int temp = arr[row - 1][j];
        for (int k = row - 1; k > 0; k--) {
            arr[k][j] = arr[k - 1][j];
        }
        arr[0][j] = temp;
        position = (position + 1) % row;
    }
}
```

You're working as a terrain analyst for a Mars exploration rover. The rover receives a grid (matrix) of terrain height values scanned from orbit. However, due to occasional solar interference, some readings show spikes (noisy values) that are unrealistically high and should be ignored.

Your task is to complete the following function/method to help the rover find a local minimum — a position where the height is lower than its adjacent neighbors (top, bottom, left, right), excluding any cells where the height is above a certain noise threshold.

To help guide the rover better, **if multiple valid local minima exist, choose the last one from the bottom right of the matrix.**

A local minimum is a cell strictly less than its up, down, left, and right neighbors (4 directions).

Ignore any cell or neighbor that is above the given noise threshold.

Input Array	Returned Array	Explanation

`matrix = [`
`[ 22, 21, 1000, 23],`
`[ 25, 17, 21, 1000],`
`[1000, 19, 18, 22],`
`[ 21, 22, 1000, 20]`
`]`
`noise_threshold = 100`
	Local minimum found at: (2, 2)	• (2, 2) has value 18.
		• Its neighbors: 21 (up), 1000 (down, ignored), 19 (left), 22 (right).
		• 18 is less than all valid neighbors and the bottom rightmost item

Python Notation	Java Notation

`def localMinima(matrix, threshold):`
`  # To DO`
	`public static void localMinima`
	`  (int[][] matrix, int threshold){`
	`  //To DO`
	`}`

```
int localRow = -1;
int localCol = -1;

for(int i=1; i<row-1; i++){                    //Oporer ar nicher row check korbe na
   for(int j=1; j<col-1; j++){                  //Oporer ar nicher column check korbe na
      if(arr[i][j] < threshold){

         if(arr[i][j] < arr[i-1][j] &&
            arr[i][j] < arr[i+1][j] &&
            arr[i][j] < arr[i][j-1] &&
            arr[i][j] < arr[i][j+1]){

            localRow = i;
            localCol = j;
         }
      }
   }
}
```

```
if(localRow != -1)
    System.out.printf("Local minimum found at (%d, %d)", localRow, localCol);
else
    System.out.println("No local minimum found.");
```

Additional

Exit Point of matrix

0	0	1	0
1	0	0	1
0	0	0	1
1	0	1	0

```
//Direction
        East= 0 [1, j+1]
        South=1 [i+1, j]
        West = 2 [i, j-1]
        North= 3 [i-1, j]


int dir= 0;
int i= 0;
int j=0;

while(true){
  dir= (dir + arr[i][j])%4;

        if(dir==0){
          j++;
        }
        else if(dir==2){
          i++;
        }
        else if(dir==3){
          j--;
        }
        else if(dir==2){
          i–;
        }
```

```
}
```

---

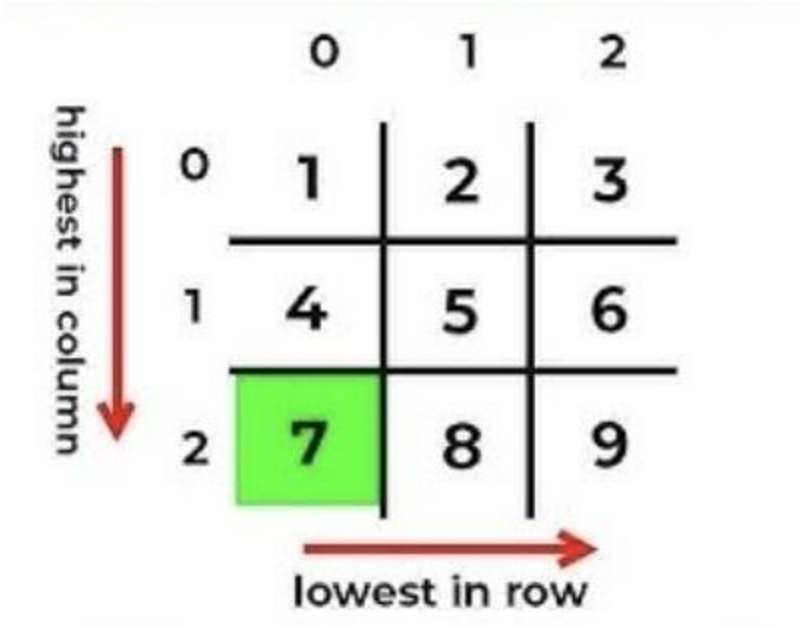
2. Write a function named **foldUp**() that takes a 2D square matrix as input and folds it along the secondary diagonal (from the top-right to the bottom-left), in-place.
For every element below the secondary diagonal, add its value to the element mirrored across the secondary diagonal, then set the original (lower) elements to zero.

- The transformation must be done in-place (no new matrix).
- If the input matrix is not square, return it unchanged.

Input	Output	Explanation
matrix= 1 2 3 4 5 6 7 8 9	10 8 3 12 5 0 7 0 0	The values below the secondary diagonal are added to their symmetric counterparts: • 9 is added to its counterpart 1, resulting in 10. • 6 is added to its counterpart 2, resulting in 8. • 8 is added to its counterpart 4, resulting in 12. Elements below the secondary diagonal are zeroed after addition.

```
for (int i = row - 1; i >= 0; i--) {
   for (int j = 0; j < col; j++) {
      if (i + j > row - 1) {          // below secondary diagonal
         arr[row - 1 - j][col - 1 - i] += arr[i][j];
         arr[i][j] = 0;
      }
   }
}
```

Saddle Point




```
for(int i = 0; i < arr.length; i++){
   int svj = 0;                          //Smallest column in the row index
//Row fixed, column check
   for(int j = 1; j < arr[0].length; j++){
        if(arr[i][j] < arr[i][svj]){
              svj= j;
        }
      }



   boolean flag = true;
//Col fixed, row change
   for(int k = 0; k < arr.length; k++){
      if(arr[k][svj] > arr[i][svj]){
            flag = false;
            break;
        }

     if(flag == true){
      System.out.println(arr[i][svj]);
       return;
      }
}
```

```
System.out.println("Invalid input");
```